📄 PAGE 2 — Set 1

Q.1 Demonstrate Blockchain [15M]

Blockchain is a distributed, decentralized digital ledger that records transactions across multiple nodes in a secure, transparent, and tamper-proof manner.

How it works (Step-by-step Demo):

1. **Transaction Initiated** — A user initiates a transaction (e.g., sending cryptocurrency).
2. **Block Created** — The transaction is grouped with others into a block.
3. **Block Broadcast** — The block is broadcast to all nodes in the P2P network.
4. **Validation** — Nodes validate the transaction using consensus mechanisms (PoW/PoS).
5. **Block Added** — Once validated, the block is added to the existing chain permanently.
6. **Transaction Complete** — The transaction is now immutable and publicly recorded.

Key Properties:

- Decentralized — No single authority controls it
- Immutable — Records cannot be altered once written
- Transparent — All participants can view the ledger
- Secure — Uses cryptographic hashing (SHA-256)

Example: Bitcoin blockchain — every BTC transaction is recorded in blocks, chained together using hashes.

Q.2 a) Ethereum Network [15M]

Ethereum is an open-source, decentralized blockchain platform that enables developers to build and deploy **smart contracts** and **decentralized applications (dApps)**.

Ethereum Networks:

Network	Type	Purpose
Mainnet	Production	Real ETH, real transactions
Ropsten	Testnet (deprecated)	Testing with PoW
Rinkeby	Testnet (deprecated)	Testing with PoA
Goerli	Testnet	Cross-client testing
Sepolia	Testnet	Current recommended testnet
Local (Ganache)	Private	Local development

Q.2 b) Where do nodes run smart contract code?

Nodes run smart contract code on the **Ethereum Virtual Machine (EVM)**. Every full node on the Ethereum network runs the EVM, which executes smart contract bytecode in a sandboxed, deterministic environment. This ensures all nodes reach the same result.

Q.2 c) Leading Open Source Blockchain Platforms

- **Ethereum** — Smart contracts, dApps (Solidity)
- **Hyperledger Fabric** — Enterprise-grade permissioned blockchain (Linux Foundation)
- **Corda** — Financial services focused (R3)
- **EOS** — High-throughput dApp platform
- **Tezos** — Self-amending blockchain
- **Stellar** — Cross-border payments
- **IOTA** — IoT-focused, DAG-based ledger

📄 PAGE 3 — Set 2

Q.1 Case Study on Cybersecurity Risk Management — Control Architecture [15M]

Case Study: Financial Institution Cybersecurity Risk Management

Overview: A bank manages sensitive financial data and must implement a comprehensive cybersecurity risk management framework.

Control Architecture:

1. Governance Layer

- Risk Management Committee oversees cybersecurity policy
- CISO (Chief Information Security Officer) leads security operations
- Policies aligned with ISO 27001, NIST Cybersecurity Framework

2. Preventive Controls

- Firewalls, IDS/IPS, endpoint protection
- Multi-Factor Authentication (MFA)
- Data encryption (AES-256, TLS 1.3)
- Role-Based Access Control (RBAC)

3. Detective Controls

- SIEM (Security Information & Event Management) tools — Splunk, IBM QRadar
- Real-time log monitoring
- Anomaly detection using AI/ML

4. Corrective Controls

- Incident Response Plan (IRP)
- Business Continuity Plan (BCP) and Disaster Recovery Plan (DRP)
- Patch management process

Supporting Processes:

- Regular vulnerability assessments and penetration testing
- Employee security awareness training
- Third-party vendor risk assessment
- Compliance audits (PCI DSS, RBI guidelines)
- Risk register maintenance — identifying, assessing, mitigating risks

---

Q.2 a) Cryptography and its role in Blockchain [15M]

Cryptography is the practice of securing information by transforming it into an unreadable format using mathematical algorithms.

Role in Blockchain:

- **Hashing (SHA-256)** — Each block contains a hash of the previous block, ensuring immutability
- **Digital Signatures (ECDSA)** — Verify identity of transaction sender without revealing private key
- **Public/Private Key Pairs** — Users have a public address and private key for signing transactions
- **Merkle Trees** — Efficiently verify large sets of transactions within a block

Q.2 b) Blocks in Blockchain

A **block** is the fundamental unit of a blockchain. Each block contains:

- **Block Header** — Hash of previous block, timestamp, nonce, Merkle root
- **Transaction Data** — List of all transactions in that block
- **Hash** — Unique cryptographic fingerprint of the block

Blocks are linked in chronological order forming a chain. Altering one block invalidates all subsequent blocks.

Q.2 c) Smart Contract and Applications

A **smart contract** is a self-executing program stored on the blockchain that automatically enforces and executes the terms of an agreement when predefined conditions are met — no intermediary needed.

Applications:

- DeFi (Decentralized Finance) — Lending, borrowing without banks
- NFTs — Ownership and transfer of digital assets
- Supply Chain — Automated payment on delivery confirmation
- Insurance — Auto-claim settlement based on data triggers
- Healthcare — Secure patient data sharing with consent
- Voting Systems — Tamper-proof digital elections

📄 PAGE 4 — Set 3

Q.1 a) PCI DSS — Payment Card Industry Data Security Standard [15M]

PCI DSS is a set of security standards developed by the PCI Security Standards Council (founded by Visa, MasterCard, Amex, Discover) to protect cardholder data during payment transactions.

12 Core Requirements:

1. Install and maintain a firewall
2. Don't use vendor-supplied defaults for passwords
3. Protect stored cardholder data
4. Encrypt transmission of cardholder data
5. Use antivirus software
6. Develop and maintain secure systems
7. Restrict access to cardholder data (need-to-know)
8. Assign unique IDs to each person with computer access
9. Restrict physical access to cardholder data
10. Track and monitor all access to network resources
11. Regularly test security systems
12. Maintain an information security policy

Q.1 b) RBI PSS — Reserve Bank of India Payment and Settlement System

RBI PSS refers to the **Payment and Settlement Systems Act, 2007**, which gives RBI the authority to regulate and supervise all payment systems in India.

Key aspects:

- RBI authorizes payment system operators
- Covers systems like RTGS, NEFT, IMPS, UPI, NACH
- Ensures safety, security, and efficiency of payment systems
- Mandates cybersecurity frameworks for payment operators
- Enforces KYC/AML compliance
- Issues directives for data localization (payment data stored in India)

Q.1 c) PCI Compliance Checklist

- ✅ Firewall installed and configured
- ✅ Unique passwords (no vendor defaults)
- ✅ Cardholder data encrypted (AES-256)
- ✅ SSL/TLS for data in transit
- ✅ Antivirus/antimalware active
- ✅ Access control on need-to-know basis
- ✅ MFA enabled for admin access
- ✅ Security logs maintained and monitored
- ✅ Quarterly vulnerability scans
- ✅ Annual penetration testing
- ✅ Incident response plan documented
- ✅ Employee training on security policies

Q.2 Case Study — Alibaba, Alipay to Ant Financial [15M]

Background: Alibaba launched **Alipay** in 2004 as an escrow payment service for its e-commerce platform Taobao, solving the trust problem between online buyers and sellers in China.

Evolution:

- **2004** — Alipay launched as basic escrow payment
- **2013** — Yu'e Bao (money market fund) launched — users earn interest on Alipay balance; became world's largest money market fund
- **2014** — Alipay expanded to offline payments via QR codes
- **2014** — Ant Financial Services Group spun off from Alibaba
- **2016** — Launched MYbank (digital bank for SMEs)
- **2020** — Rebranded as **Ant Group**

Key Innovations:

- QR code-based mobile payments
- Credit scoring using big data — **Zhima (Sesame) Credit**
- AI-based fraud detection
- Blockchain for supply chain finance

Security Measures:

- Real-time risk monitoring — 110ms to detect fraud
- Biometric authentication (face, fingerprint)
- Encrypted transaction data
- Anti-phishing and anti-fraud systems

Impact: Ant Financial transformed China into a nearly cashless society and became a global model for mobile-first financial services.

📄 PAGE 5 — Set 4

Q.1 a, b, c) — Same as Page 3 Q.2 (Cryptography, Blocks, Smart Contracts)

(Refer to Page 3 solutions above)

Q.2 Case Study — Fintech in Singapore [15M]

Overview: Singapore is one of the world's leading Fintech hubs, driven by strong government support, robust regulatory frameworks, and a pro-innovation environment.

Key Drivers:

1. Regulatory Support

- **Monetary Authority of Singapore (MAS)** acts as both central bank and financial regulator
- MAS launched the **Fintech Regulatory Sandbox** — allows companies to test innovations in a controlled environment
- Smart Financial Centre vision — digitizing financial industry

2. Key Fintech Innovations in Singapore:

- **PayNow** — Real-time peer-to-peer fund transfers using mobile/NRIC number
- **SGFinDex** — Consent-based financial data sharing across institutions
- Digital banking licenses issued to non-bank entities (GrabBank, SEA/MariBank)
- Blockchain for trade finance — **Contour** platform

3. Cybersecurity in Fintech:

- MAS Technology Risk Management (TRM) Guidelines

- Mandatory cyber hygiene practices for financial institutions
- Cyber Security Agency (CSA) of Singapore coordinates national cybersecurity
- Regular industry-wide cyber exercises (e.g., Operation Cyber Star)

4. Collaboration:

- **Singapore Fintech Festival** — World's largest Fintech event
- MAS partners with global regulators through Innovation Bridges
- Institutes like NUS, NTU contribute Fintech R&D

Outcome: Singapore ranks consistently in top 3 global Fintech hubs due to its balanced approach of innovation + regulation + security.

📄 PAGE 6 — Set 5

Q.1 a) Security in CI/CD Pipeline [15M]

CI/CD (Continuous Integration/Continuous Deployment) pipeline security involves integrating security checks at every stage of development:

Steps to implement security:

1. **Code Stage** — Static Application Security Testing (SAST) — tools like SonarQube, Checkmarx
2. **Build Stage** — Software Composition Analysis (SCA) — check open-source libraries for vulnerabilities (Snyk, OWASP Dependency-Check)
3. **Test Stage** — Dynamic Application Security Testing (DAST) — OWASP ZAP, Burp Suite
4. **Deploy Stage** — Container security scanning (Trivy, Clair for Docker images)
5. **Runtime** — RASP (Runtime Application Self-Protection), monitoring with SIEM

Q.1 b) DevOps vs DevSecOps

Aspect	DevOps	DevSecOps
Focus	Speed + collaboration	Speed + security + collaboration
Security	Added at end	Integrated throughout (Shift Left)
Teams	Dev + Ops	Dev + Sec + Ops
Testing	Functional testing	Functional + security testing
Tools	Jenkins, Docker, K8s	Jenkins + SAST/DAST/SCA tools
Goal	Fast delivery	Fast + secure delivery

Q.1 c) Common Security Tools in DevSecOps

- **SAST:** SonarQube, Checkmarx, Veracode
- **DAST:** OWASP ZAP, Burp Suite
- **SCA:** Snyk, WhiteSource, OWASP Dependency-Check
- **Container Security:** Trivy, Aqua Security, Clair
- **Secrets Management:** HashiCorp Vault, AWS Secrets Manager
- **IaC Security:** Checkov, tfsec (for Terraform)
- **SIEM:** Splunk, ELK Stack

Q.2 a) PCI DSS & b) RBI PSS

(Refer to Page 4 Q.1 solutions above)

📄 PAGE 7 — Set 6

Q.1 RBI PSS — Compliance within Fintech [15M]

(Refer to Page 4 Q.1b solution above for RBI PSS details)

Additional compliance aspects:

- **BBPS (Bharat Bill Payment System)** — Interoperable bill payment under RBI
- **PPI (Prepaid Payment Instruments)** regulations — wallets, gift cards
- **Tokenization** — RBI mandated card tokenization for merchant payments
- **Grievance Redressal** — Financial institutions must have digital complaint mechanisms
- Penalties for non-compliance — cancellation of authorization

---

Q.2 a) DevSecOps in Fintech

DevSecOps integrates security practices within the DevOps process, making security a shared responsibility across Dev, Sec, and Ops teams.

In Fintech context:

- Protects sensitive financial data during development
- Reduces cost of fixing vulnerabilities (cheaper to fix early)
- Meets regulatory compliance requirements (PCI DSS, RBI, GDPR)
- Continuous security testing in pipelines

Q.2 b) CI/CD Introduction

CI (Continuous Integration) — Developers frequently merge code into a shared repository; automated builds and tests run on each commit.

CD (Continuous Deployment/Delivery) — Code is automatically deployed to staging or production after passing tests.

CI/CD Pipeline Stages: Code → Build → Test → Release → Deploy → Monitor

Q.2 c) SSDLC Introduction

SSDLC (Secure Software Development Life Cycle) integrates security at every phase of the SDLC:

Phase	Security Activity
Requirements	Security requirements, threat modeling
Design	Secure architecture review
Development	Secure coding standards, code review
Testing	SAST, DAST, penetration testing
Deployment	Hardening, secure configuration
Maintenance	Patch management, monitoring

■ PAGE 8 — Set 7

Q.1 Software Composition Analysis (SCA) — Security Case Study [15M]

SCA is the process of identifying and analyzing open-source components (libraries, frameworks, dependencies) used in an application to detect known vulnerabilities, license risks, and outdated packages.

Security Case Study — Financial App:

Scenario: A Fintech company builds a payment app using open-source libraries. Without SCA, a vulnerable version of Apache Log4j (Log4Shell — CVE-2021-44228) is included.

Without SCA: Attackers exploit Log4Shell to gain remote code execution, exposing millions of financial records.

With SCA:

- SCA tool (Snyk) scans dependencies during CI/CD pipeline
- Detects Log4j 2.14.1 has critical CVE
- Automatically flags and blocks deployment

- Suggests upgrading to Log4j 2.17.1 (patched version)
- Developer fixes before production — zero breach

Popular SCA Tools:

- Snyk
- WhiteSource (Mend)
- OWASP Dependency-Check
- Black Duck
- Sonatype Nexus IQ

Benefits in Fintech:

- Meets PCI DSS Requirement 6 (secure systems)
- Reduces supply chain attack risk
- Automates license compliance
- Integrates seamlessly into DevSecOps pipelines

Q.2 a) Challenge when applying Shift Left approach

Main Challenge: Cultural resistance — developers are not traditionally trained in security. Shifting security to early stages requires:

- Training developers in secure coding
- Integrating new tools into existing workflows
- Managing false positives from automated scanners
- Balancing speed of delivery with security thoroughness
- Getting buy-in from management for increased upfront costs

Q.2 b) Why DevOps recommend Shift Left testing

- Bugs and vulnerabilities are **10x cheaper** to fix in development than in production
- Faster feedback loop — developers fix issues immediately
- Prevents security from becoming a bottleneck at release
- Improves overall code quality
- Aligns with agile and continuous delivery principles
- Reduces risk of breaches in production

Q.2 c) Main focus of automating security checks in Shift Left

The main focus is **early and continuous detection of vulnerabilities** through:

- Automated SAST on every code commit
- SCA for open-source dependency scanning
- Automated unit tests with security assertions

- Secret scanning (no hardcoded passwords/API keys)
- IaC (Infrastructure as Code) security scanning
- Integration into IDE plugins so developers see issues in real-time

---

■ PAGE 9 — Set 8

Q.1 PSD2 — Payment Services Directive 2 — Security Case Study [15M]

PSD2 is a European Union directive that regulates payment services and payment service providers, with a strong focus on security and open banking.

Security Case Study — Open Banking Implementation:

Scenario: A UK bank implements PSD2 compliance for its mobile banking app.

Key Security Implementation:

1. **Strong Customer Authentication (SCA)** — Multi-factor authentication mandatory for online payments above €30
    - Something you know (PIN/password)
    - Something you have (phone/token)
    - Something you are (biometric)
2. **Open API Security** — Bank exposes APIs to Third Party Providers (TPPs); secured using OAuth 2.0 + OpenID Connect
3. **Transaction Monitoring** — Real-time fraud detection to identify anomalies
4. **Outcome:** Reduced card fraud by 72% in EU after PSD2 implementation; enabled innovation through open banking APIs

---

Q.2 a) Standards of eIDAS

eIDAS (Electronic Identification, Authentication and Trust Services) is an EU regulation establishing standards for:

- Electronic identification (eID)
- Electronic signatures (Simple, Advanced, Qualified)
- Electronic seals, timestamps, registered delivery services
- Website authentication certificates

Three levels of assurance: Low, Substantial, High

Q.2 b) Use Case of eIDAS

- **Cross-border digital identity** — EU citizen uses national eID to access government services in another EU country
- **Qualified Electronic Signature (QES)** — Legally equivalent to handwritten signature for contracts
- **e-Invoicing** — Secure electronic invoices for businesses
- **Healthcare** — Patients access medical records across EU countries using eID

Q.2 c) Security Levels of eIDAS

Level	Description	Example
Low	Minimal confidence in identity	Username/password
Substantial	Moderate confidence, at least one authentication factor	Smart card, OTP
High	High confidence, multi-factor, hardware-based	Biometric + hardware token

---

📄 PAGE 10 — Set 9

Q.1 eIDAS — Security Case Study [15M]

(Extended from Page 9)

Case Study — Cross-Border Digital Identity in EU:

Scenario: A Portuguese citizen wants to access German government tax services online.

Without eIDAS: Would need in-person verification or separate German digital ID.

With eIDAS:

1. Portuguese citizen logs in using Portuguese national eID (Citizen Card)
2. eIDAS node in Portugal sends authenticated identity to Germany
3. German service accepts the identity under eIDAS mutual recognition
4. Citizen accesses the service seamlessly

Security Features:

- End-to-end encrypted identity assertions
- Qualified Trust Service Providers (QTSPs) audited annually
- Non-repudiation through qualified electronic signatures
- Revocation mechanisms for compromised eIDs

Key Components:

- **eID** — Electronic identity for authentication

- **eSignature** — Digital signing of documents
- **eSeal** — Organizational equivalent of eSignature
- **eTimestamp** — Proof of data existence at a point in time
- **eDelivery** — Secure registered electronic delivery
- **Website Authentication** — SSL/TLS certificates verifying website identity

---

Q.2 a) Requirements for PSD2 Payment

- Strong Customer Authentication (SCA) with at least 2 factors
- Dynamic linking — authentication code linked to specific amount and payee
- Transaction risk analysis exemptions (low-risk, low-value transactions)
- Secure communication channels between banks and TPPs

Q.2 b) How PSD2 affects banks

- Banks must open APIs to licensed Third Party Providers (Account Information Service Providers — AISP, Payment Initiation Service Providers — PISP)
- Banks cannot charge TPPs for data access
- Must implement SCA for customer authentication
- Increased competition from Fintechs using bank data
- Need to invest in API infrastructure and security

Q.2 c) Purpose of PSD2

- Increase competition and innovation in payment services
- Improve consumer protection and rights
- Enhance security of electronic payments (SCA mandate)
- Enable open banking ecosystem
- Reduce payment fraud across EU
- Harmonize payment regulations across EU member states

📄 PAGE 11 — Set 10

Q.1 Shift Left Principle in Fintech — Security Case Study [15M]

Shift Left means moving security testing and practices earlier (to the left) in the software development lifecycle, rather than testing only at the end.

Case Study — Fintech Payment Gateway:

Scenario: A Fintech startup building a payment gateway adopts Shift Left security.

Traditional approach (without Shift Left):

- Security tested only before production
- Critical vulnerability (SQL injection in payment form) found during final testing
- 3-week delay, $50,000 rework cost

With Shift Left:

1. **Requirements phase** — Threat modeling identifies SQL injection as risk
2. **Development** — IDE plugin (SonarLint) flags SQL injection pattern in real-time
3. **CI pipeline** — SAST tool (Checkmarx) catches vulnerability on commit
4. **Fixed in 30 minutes** — No delay, zero extra cost

Benefits in Fintech:

- Early vulnerability detection reduces breach risk
- Faster time-to-market without sacrificing security
- PCI DSS compliance achieved more efficiently
- Developer security awareness improves over time
- Cost savings — NIST estimates 100x more expensive to fix in production

---

Q.2 a, b, c) PSD2 Requirements, Effects, Purpose

(Same as Page 10 Q.2 — refer above)

■ PAGE 12 — Set 11

Q.1 a) Risks of using PSD2 [15M]

- **Data Privacy risks** — Third parties accessing sensitive financial data
- **API Security vulnerabilities** — Poorly secured open banking APIs can be exploited
- **Phishing attacks** — Fraudsters impersonate TPPs to steal credentials
- **Third-party risk** — TPPs may have weaker security than banks
- **Consent management issues** — Users may not fully understand data sharing permissions
- **Increased attack surface** — More API endpoints = more potential vulnerabilities

Q.1 b) Functions of PSD2

- Regulate payment service providers across EU

- Mandate SCA for electronic payments
- Enable open banking via API access
- Protect consumers from unauthorized transactions
- Reduce payment fraud
- Create level playing field between banks and Fintechs
- Mandate liability rules — unauthorized payment liability on bank

Q.1 c) Scope of PSD2

- Applies to all payment services within EU/EEA
- Covers credit transfers, direct debits, card payments
- Applies to banks, e-money institutions, payment institutions
- Covers both consumer and business payments
- Applies when at least one party is in EU/EEA
- Includes currency conversions within EU

Q.2 Shift Left Principle

(Refer to Page 11 Q.1 solution above)

📄 PAGE 13 — Set 12

Q.1 a, b, c) Shift Left Questions

(Refer to Page 8 Q.2 a, b, c solutions)

Q.2 Case Study on Fintech in Singapore

(Refer to Page 5 Q.2 solution)

📄 PAGE 14 — Set 13

Q.1 Case Study — Alibaba, Alipay to Ant Financial

(Refer to Page 4 Q.2 solution — full detailed case study)

Q.2 DevSecOps, CI/CD, SSDLC

(Refer to Page 7 Q.2 solutions)

■ PAGE 15 — Set 14

Q.1 Case Study on Cybersecurity Risk Management

(Refer to Page 3 Q.1 solution)

Q.2 a) Mobile Banking vs Internet Banking — Which is more secure? [15M]

Mobile banking is generally considered more secure due to:

- Biometric authentication (fingerprint/face ID)
- App-level encryption
- Device binding — app tied to specific device
- Harder to spoof than browser-based sessions
- Push notifications for real-time transaction alerts

However, internet banking has:

- Larger screen for reviewing transactions
- More mature browser security (HTTPS, HSTS)
- Risks: phishing websites, keyloggers

Verdict: Mobile banking is marginally more secure for everyday transactions due to device-level controls.

Q.2 b) How Mobile Banking Works Without Internet

- **USSD (Unstructured Supplementary Service Data)** — Dial *99# on any phone, access banking services via GSM network without internet (supported by all Indian banks via NPCI)
- **SMS Banking** — Send/receive transaction info via SMS
- **IVR (Interactive Voice Response)** — Call bank, navigate menu for balance check, mini statements
- **Offline UPI** — UPI Lite introduced by NPCI for offline payments up to ₹500 using device wallet
- **NFC Payments** — Near Field Communication for tap-to-pay without internet

Q.2 c) Technology used in UPI

UPI (Unified Payments Interface) uses:

- **IMPS (Immediate Payment Service)** — Underlying real-time settlement infrastructure
- **RESTful APIs** — Communication between PSPs and NPCI
- **Two-factor authentication** — Device binding + UPI PIN
- **128-bit SSL/TLS encryption** — Secure data transmission
- **Virtual Payment Address (VPA)** — Unique identifier (yourname@bankname) instead of account number
- **NPCI's UPI switch** — Central routing infrastructure
- **ISO 8583** — Financial transaction message standard
- **QR Code (BharatQR)** — For merchant payments

📄 PAGE 16 — Set 15

Q.1 a) Key Components of eIDAS [15M]

1. **Electronic Identification (eID)** — Digital identity schemes for authentication
2. **Electronic Signatures** — Simple, Advanced (AES), Qualified (QES)
3. **Electronic Seals** — For organizations (equivalent to company stamp)
4. **Electronic Time Stamps** — Proof of data existence at specific time
5. **Electronic Registered Delivery Services (ERDS)** — Secure equivalent of registered mail
6. **Website Authentication** — Qualified certificates for website identity

Q.1 b) Types of Trust Services under eIDAS

- **Qualified Electronic Signature (QES)** — Highest legal validity
- **Advanced Electronic Signature (AES)** — Uniquely linked to signatory
- **Qualified Electronic Seal** — For organizational data integrity
- **Qualified Timestamp** — Certified time proof
- **Qualified Web Authentication Certificate** — Website identity verification
- **Qualified Registered Delivery** — Secure data exchange with evidence

Q.1 c) How eIDAS ensures security

- **Qualified Trust Service Providers (QTSPs)** are audited and listed in national trusted lists
- Strong identity proofing before issuing eIDs
- Hardware Security Modules (HSMs) for key storage
- Certificate Revocation Lists (CRL) and OCSP for real-time validation

- Legal framework — QES has same legal effect as handwritten signature across all EU states
- Cross-border interoperability through eIDAS nodes

Q.2 SCA Case Study

(Refer to Page 8 Q.1 solution)

■ PAGE 17 — Set 16

Q.1 a) Government Initiatives supporting Fintech in Singapore [15M]

- **MAS Fintech Regulatory Sandbox** — Safe space to test innovations
- **Financial Sector Technology & Innovation (FSTI) Scheme** — MAS grants for Fintech R&D
- **Singapore Fintech Festival** — Annual global event since 2016
- **Smart Nation Initiative** — Government-wide digital transformation
- **Digital Banking Licenses** — Issued to non-bank Fintechs (Grab, Sea)
- **SGFinDex** — Public digital infrastructure for financial data sharing
- **MAS API Playbook** — Encourages open banking

Q.1 b) Prominent Fintech Innovations in Singapore

- **PayNow** — Instant P2P and P2B fund transfer via mobile/NRIC
- **SGQR** — Unified QR code payment standard
- **SGFinDex** — Consent-based financial data aggregation
- **Project Ubin** — Blockchain-based SGD (MAS experiment)
- **Project Guardian** — DeFi experiments in institutional finance
- Digital-only banks — GrabBank (GXS), MariBank (Sea)
- Robo-advisors — StashAway, Syfe

Q.1 c) Singapore's approach to Cybersecurity in Fintech

- MAS **Technology Risk Management (TRM) Guidelines** — mandatory for all financial institutions
- **Cyber Hygiene Notice** — Minimum security standards (MFA, patching, malware protection)
- **Cyber Security Agency (CSA)** — National coordination of cybersecurity
- **Financial Services-ISAC** — Threat intelligence sharing
- Regular industry-wide **cyber war gaming exercises**

- **RED Team assessments** required for critical infrastructure

---

Q.2 PCI DSS and RBI PSS

(Refer to Page 4 Q.1 solution)

■ PAGE 18 — Set 17

Q.1 a) Challenges faced by Fintech companies in Singapore [15M]

- **Regulatory compliance** — Navigating MAS requirements while scaling
- **Cybersecurity threats** — Phishing, API attacks, data breaches
- **Talent shortage** — Limited supply of skilled Fintech/security professionals
- **Legacy system integration** — Connecting with traditional bank infrastructure
- **Customer trust** — Building trust in digital-only financial services
- **Cross-border regulatory fragmentation** — Different rules in different ASEAN countries
- **High cost of compliance** — AML, KYC, data protection requirements

Q.1 b) Singapore's collaboration between Fintechs and Financial Institutions

- **MAS Innovation Lab** — Banks collaborate with startups on proof-of-concepts
- **Elevandi** — MAS-established body to facilitate public-private collaboration
- **API Marketplace** — Banks publish APIs for Fintechs to build on
- **DBS, OCBC, UOB accelerator programs** — Banks fund and mentor Fintech startups
- **Project Ubin and Guardian** — Banks and Fintechs jointly explore blockchain/DeFi
- **Singapore Fintech Festival** — Platform for banks and startups to meet and partner

---

Q.2 a) How Blockchain works

(Refer to Page 2 Q.1)

Q.2 b) Key components of Blockchain

- **Node** — Participant in the network

- **Block** — Container of transaction data
- **Chain** — Linked sequence of blocks via hashes
- **Transaction** — Unit of data recorded
- **Hash** — Cryptographic fingerprint of block
- **Consensus Mechanism** — PoW, PoS, DPoS — rules for agreement
- **Merkle Tree** — Efficient transaction verification structure
- **Smart Contract** — Self-executing code on blockchain
- **Wallet** — Public/private key pair for accessing blockchain

Q.2 c) Role of Cryptography in Blockchain

(Refer to Page 3 Q.2a)

📄 PAGE 19 — Set 18

Q.1 eIDAS Case Study

(Refer to Page 10 Q.1 for detailed case study)

Q.2 a) Common risks that SCA addresses [15M]

- **Vulnerable open-source libraries** — Known CVEs in dependencies (e.g., Log4Shell)
- **Outdated components** — Libraries with unpatched vulnerabilities
- **License compliance risks** — GPL license in proprietary code
- **Transitive dependencies** — Vulnerabilities in indirect/nested dependencies
- **Supply chain attacks** — Malicious code injected into popular packages (e.g., SolarWinds-style)
- **Dependency confusion attacks** — Malicious packages with similar names to internal ones

Q.2 b) How SCA improves security in DevOps/CI/CD

- Scans every dependency on code commit automatically
- Blocks deployment if critical CVE detected
- Generates Software Bill of Materials (SBOM) for full visibility
- Integrates with GitHub, GitLab, Jenkins natively
- Provides automated fix suggestions (upgrade to patched version)
- Continuously monitors for newly discovered vulnerabilities in production

Q.2 c) Popular SCA Tools

Tool	Company	Key Feature
Snyk	Snyk Ltd	Developer-first, IDE + CI integration
WhiteSource (Mend)	Mend	License + security scanning
Black Duck	Synopsys	Comprehensive open-source management
OWASP Dependency-Check	OWASP	Free, open-source
Sonatype Nexus IQ	Sonatype	Deep ecosystem insights
JFrog Xray	JFrog	Deep binary scanning

■ PAGE 20 — Set 19

Q.1 a, b, c) Singapore Fintech Challenges, Collaboration, and Lessons for Other Countries [15M]

(a and b — Refer to Page 18 Q.1)

c) How other countries can learn from Singapore's Fintech success:

- **Establish a regulatory sandbox** — Allow innovations to be tested safely before full regulation
- **Create a dedicated Fintech regulator/innovation office** within central bank
- **Invest in digital infrastructure** — Real-time payment systems (like PayNow/UPI)
- **Issue digital banking licenses** to non-bank entities to foster competition
- **Public-private collaboration** — Government co-invest in Fintech R&D
- **Cybersecurity-first approach** — Mandate security baselines (like TRM guidelines)
- **Talent development** — Government-funded programs for Fintech/cybersecurity skills
- **Open banking framework** — Standardize APIs for data sharing

India's parallel: India's UPI, Jan Dhan, Aadhaar stack (JAM Trinity) mirrors Singapore's approach and has become a global benchmark.

Q.2 eIDAS Case Study

(Refer to Page 10 Q.1)

■ PAGE 21 — Set 20

Q.1 PCI DSS and RBI PSS

(Refer to Page 4 Q.1 solution)

Q.2 SCA Case Study

(Refer to Page 8 Q.1 solution)

📄 PAGE 22 — Set 21

Q.1 a) Key benefits of implementing Shift Left in Fintech [15M]

- **Cost reduction** — Fixing vulnerabilities in development costs 100x less than in production
- **Faster delivery** — Security integrated into pipeline, no last-minute bottlenecks
- **Regulatory compliance** — Continuous security = easier PCI DSS, RBI compliance
- **Improved developer security awareness** — Developers become security-conscious
- **Reduced breach risk** — Vulnerabilities caught before reaching production
- **Customer trust** — Fewer incidents = stronger reputation

Q.1 b) Real-world example of Shift Left in Fintech

Example — PayPal: PayPal implemented Shift Left by:

- Integrating SAST into developer IDEs
- Running automated security tests on every pull request
- Training all engineers in OWASP Top 10
- Result: 75% reduction in security vulnerabilities reaching production
- Faster release cycles with maintained security posture

Q.1 c) How Shift Left enhances Dev-Security collaboration

- **Shared responsibility model** — Security is everyone's job, not just security team's
- **Developers get immediate feedback** — IDE plugins show security issues in real-time
- **Security teams become advisors** — Rather than gatekeepers blocking releases
- **Shared tooling** — Both teams use same dashboards (e.g., Snyk, Checkmarx)
- **Joint threat modeling** — Security architects involved from design phase
- **Security champions program** — Designated developer in each team who bridges Dev and Sec

---

Q.2 Alibaba, Alipay to Ant Financial

(Refer to Page 4 Q.2 solution)

📄 PAGE 23 — Set 22

Q.1 a) Why eIDAS is important for security [15M]

- Provides **legally recognized** digital identity across 27 EU member states
- Eliminates need for insecure paper-based identity verification
- Ensures **non-repudiation** — Qualified signatures cannot be denied
- Establishes **liability** — If a QTSP issues a faulty certificate, they are legally liable
- Reduces identity fraud and impersonation in digital transactions
- Enables secure cross-border digital services

Q.1 b) How eIDAS supports electronic identification

- Member states can notify their national eID schemes to EU
- Notified eID schemes are accepted by all public sector bodies across EU
- Uses standardized assurance levels (Low/Substantial/High)
- Cross-border authentication via eIDAS interoperability framework
- Citizens can prove identity digitally without physical documents
- Example: Estonian e-Residency — 99% of government services accessible digitally

Q.1 c) Key components of trust services under eIDAS

(Refer to Page 16 Q.1b — Types of Trust Services)

Q.2 SCA Case Study

(Refer to Page 8 Q.1)

📄 PAGE 24 — Set 23

Q.1 PCI DSS and RBI PSS

(Refer to Page 4 Q.1)

Q.2 How Blockchain works, Key Components, Cryptography role

(Refer to Pages 2 and 18 solutions)

■ PAGE 25 — Set 24

Q.1 SCA Case Study

(Refer to Page 8 Q.1 — full case study)

Q.2 a) DevSecOps in Fintech [15M]

DevSecOps = Development + Security + Operations integrated into one continuous process.

In Fintech:

- Security is embedded from day one of development
- Prevents vulnerabilities in payment systems, APIs, user data
- Regulatory compliance (PCI DSS, RBI, GDPR) built into pipeline
- Teams include developers, security engineers, and operations together
- **Key practices:** Threat modeling, SAST/DAST, SCA, secrets scanning, container security

Q.2 b) CI/CD Explained

CI (Continuous Integration):

- Developers push code to shared repository (GitHub/GitLab) multiple times/day
- Automated build and test triggered on each push
- Catches integration bugs early

CD (Continuous Delivery/Deployment):

- Every passing build is automatically deployable (Delivery) or deployed (Deployment)
- Reduces manual release effort
- Ensures always-releasable codebase

Pipeline: Code Commit → Build → Unit Tests → Security Scan → Staging → Integration Tests → Production Deploy → Monitor

Q.2 c) SSDLC Explained

SSDLC (Secure Software Development Life Cycle) adds security activities to each traditional SDLC phase:

SDLC Phase	SSDLC Addition
Planning	Security requirements gathering
Analysis	Risk assessment, threat modeling
Design	Secure architecture, attack surface analysis
Implementation	Secure coding standards, peer code review
Testing	SAST, DAST, pen testing, fuzz testing
Deployment	Hardening, configuration review
Maintenance	Vulnerability monitoring, patching

Popular SSDLC Models: Microsoft SDL, OWASP SAMM, BSIMM